

MPI y CUDA

TeamNo6 — Parcial 3

David Bayona y Felipe Moncada

Computación Paralela

1. Introducción

La computación paralela es una de las disciplinas más relevantes de la ciencia de la computación moderna. Su premisa fundamental es simple pero poderosa: en lugar de resolver un problema de forma secuencial, dividirlo en subproblemas que puedan ejecutarse simultáneamente en múltiples unidades de procesamiento. Esta idea, que parece intuitiva, encierra una riqueza técnica enorme que abarca desde el diseño de algoritmos eficientes hasta la gestión de memoria, la sincronización de procesos y la tolerancia a fallos.

El presente ensayo analiza los conceptos clave detrás de dos de las tecnologías paralelas más utilizadas en entornos de High Performance Computing (HPC): la interfaz de paso de mensajes MPI (Message Passing Interface) y la arquitectura de cómputo en GPU mediante CUDA (Compute Unified Device Architecture). Para ilustrar estos conceptos, se toma como caso de estudio la multiplicación de matrices cuadradas $N \times N$, un problema clásico con complejidad $O(N^3)$ que escala rápidamente y representa un benchmark natural para evaluar el rendimiento paralelo.

2. El Problema de la Escalabilidad Computacional

Antes de explorar las soluciones paralelas, conviene entender por qué la computación secuencial resulta insuficiente para ciertos problemas. Multiplicar dos matrices de $10,000 \times 10,000$ elementos implica del orden de 10^{12} operaciones de punto flotante. En un procesador moderno capaz de ejecutar 10^9 operaciones por segundo, esto requeriría aproximadamente 1,000 segundos, es decir, más de 16 minutos para una sola multiplicación.

En aplicaciones reales como simulaciones climáticas, renderizado de imágenes 3D, entrenamiento de redes neuronales o análisis genómico, matrices de esta escala son comunes, y los tiempos de ejecución secuenciales se vuelven prohibitivos.

La respuesta de la industria y la academia ha sido el HPC: sistemas con cientos, miles o incluso millones de procesadores trabajando en conjunto. Sin embargo, paralelizar un algoritmo no consiste simplemente en ejecutarlo en más procesadores. Requiere rediseñar el algoritmo, gestionar la comunicación entre procesos, balancear la carga de trabajo y garantizar la correctitud del resultado. Es aquí donde entran en juego MPI y CUDA, cada uno con su modelo de programación y sus casos de uso óptimos.

3. MPI: Paralelismo por Paso de Mensajes

3.1. Modelo de Programación

MPI es un estándar de comunicación para programas que se ejecutan en múltiples procesos, potencialmente distribuidos en distintas máquinas físicas conectadas por una red. A diferencia del paralelismo de memoria compartida, en MPI cada proceso tiene su propio espacio de memoria privado y los datos se intercambian explícitamente mediante mensajes.

El modelo de programación MPI sigue el paradigma SPMD (Single Program, Multiple Data): todos los procesos ejecutan el mismo programa, pero cada uno opera sobre un subconjunto diferente de los datos.

En la multiplicación de matrices, esto se traduce en que el proceso raíz (rank 0) posee inicialmente las matrices A y B completas, las distribuye entre los procesos usando operaciones colectivas, cada proceso calcula su porción de la matriz resultado C , y finalmente el proceso raíz recolecta todos los fragmentos.

3.2. Operaciones Colectivas y su Eficiencia

Una de las contribuciones más importantes de MPI son las operaciones colectivas. En lugar de enviar mensajes punto a punto desde el proceso raíz a cada uno de los demás procesos, las operaciones colectivas explotan topologías de comunicación eficientes como árboles binarios, reduciendo significativamente la latencia.

En el proyecto de multiplicación de matrices se utilizan tres operaciones colectivas:

- `MPI_Bcast`, que transmite la matriz B completa a todos los procesos.
- `MPI_Scatter`, que divide las filas de A entre los procesos.
- `MPI_Gather`, que recolecta los resultados parciales para reconstruir la matriz final.

Estas operaciones reducen el tráfico de red y simplifican el diseño del algoritmo paralelo.

3.3. Tiling Local en Cada Proceso

Paralelizar el cómputo no es suficiente si cada proceso realiza cálculos de manera ineficiente. Por ello, dentro de cada proceso MPI se aplica la técnica de *tiling* o *blocking*.

La idea consiste en dividir las matrices en bloques de tamaño `TILE_SIZE × TILE_SIZE`. Cuando un tile cabe completamente en caché, los accesos a memoria son más rápidos y se reducen drásticamente los cache misses.

Para la multiplicación de matrices, el tiling mejora la localidad espacial y temporal de los datos, optimizando el uso de la jerarquía de memoria de la CPU.

4. CUDA: Paralelismo Masivo en GPU

4.1. Arquitectura GPU vs CPU

Mientras que una CPU moderna posee pocos núcleos altamente optimizados para latencia baja, una GPU contiene miles de núcleos simples optimizados para throughput.

Esta diferencia arquitectónica hace que las GPUs sean ideales para problemas altamente paralelos donde se ejecuta la misma operación sobre grandes volúmenes de datos, exactamente el caso de la multiplicación de matrices.

CUDA es el modelo de programación desarrollado por NVIDIA para aprovechar esta arquitectura.

4.2. Shared Memory y Sincronización

La técnica de tiling en CUDA se complementa con el uso de *shared memory*, una memoria rápida compartida por todos los hilos de un bloque.

El algoritmo carga colaborativamente tiles de las matrices A y B en shared memory. Cada hilo carga una parte de los datos y luego todos cooperan para calcular un sub-bloque de la matriz resultado.

La sincronización es crítica en este proceso. La instrucción:

`__syncthreads()`

actúa como una barrera que obliga a todos los hilos del bloque a sincronizarse antes de continuar.

Sin estas barreras podrían aparecer condiciones de carrera y resultados incorrectos.

5. Validación de Correctitud

Un aspecto fundamental de la computación paralela es garantizar que los resultados obtenidos sean correctos.

Para ello, se implementó una versión secuencial conocida como *golden reference*. Tanto la versión MPI como la versión CUDA comparan sus resultados contra esta referencia usando una tolerancia:

$$\epsilon = 10^{-3}$$

La tolerancia es necesaria debido a que la aritmética de punto flotante no es completamente asociativa y pequeñas diferencias numéricas pueden aparecer debido al orden de las operaciones.

6. Análisis de Complejidad

La multiplicación tradicional de matrices tiene complejidad temporal:

$$O(N^3)$$

En MPI, usando p procesos, el trabajo computacional se reduce aproximadamente a:

$$O\left(\frac{N^3}{p}\right)$$

aunque aparece un costo adicional de comunicación debido al intercambio de datos.

En CUDA, miles de hilos trabajan simultáneamente sobre distintos elementos de la matriz resultado, reduciendo considerablemente el tiempo total de ejecución.

Además, el uso de shared memory reduce los accesos a memoria global y mejora significativamente el rendimiento.

7. Conclusiones

La computación paralela se ha convertido en una necesidad para resolver problemas científicos y computacionales de gran escala. MPI y CUDA representan dos paradigmas complementarios: el primero orientado a sistemas distribuidos y el segundo al paralelismo masivo en GPU.

La multiplicación de matrices permite observar claramente conceptos fundamentales como distribución de datos, sincronización, jerarquía de memoria y optimización mediante tiling.

Asimismo, el proyecto demuestra que el rendimiento no depende únicamente de la cantidad de procesadores disponibles, sino también de la forma en que se gestionan la memoria y la comunicación.